Block Matchers

Block matchers observe a block of code and are used to specify a side effect that occurs when the block runs. They take the form:

```
expect { some_code }.to matcher
```

As with value matchers, block matchers can be negated by using not_to or to_not rather than to.

Mutation

The change matcher captures a value before running the block (old_value) and again after running the block (new_value). The value can be specified in two ways:

```
expect { do_something }.to change(obj, :attr)
# or
expect { do_something }.to change { obj.attr }
```

It supports a rich fluent interface for specifying further details about the mutation:

Matcher	Passes if…
change { }	old_value != new_value
change { }.by(x)	(new_value - old_value) == x
change { }.by_at_least(x)	(new_value - old_value) >= x
change { }.by_at_most(x)	(new_value - old_value) <= x
change { }.from(x)	old_value != new_value && old_value == x
change { }.to(y)	old_value != new_value && new_value == y
change { }.from(x).to(y)	old_value != new_value && old_value == x && new_value == y